

TP03 Report — Dependency Injection & Circular Dependency in NestJS

□ Practice 3 Overview

This practice focuses on **Dependency Injection (DI)** and **Circular Dependency** in NestJS. We learn how to:

- Build **provider chains** — one service injecting another service
- Create a **circular dependency** on purpose — to understand the problem
- Fix circular dependency using **forwardRef()** — the quick fix
- Fix circular dependency using **architectural refactoring** — the proper solution

Updated Project Structure (after TP03)

```
src/
├── main.ts                # App entry point
├── app.module.ts          # Root module – now imports Notifications,
Orders, Core              # Root controller (GET /)
├── app.controller.ts      # Root service
├── app.service.ts
├── common/
│   ├── guards/
│   │   └── api-key.guard.ts    # API Key guard (from TP02)
│   └── interceptors/
│       └── logging.interceptor.ts # Logging interceptor (from TP02)
├── core/
│   └── core.module.ts          # □ NEW – Part D: Token-based abstraction
│                               # CoreModule – provides EVENT_PUBLISHER
token
├── tokens.ts               # Symbol token definition
├── database/
│   └── entities/
│       ├── receipts.entity.ts  # Receipt entity (from TP02)
│       └── order.entity.ts     # □ NEW – Order entity for PostgreSQL
├── notifications/
│   ├── notifications.module.ts # □ NEW – Part A: Notification service
│   └── notifications.service.ts # NotificationsModule
│                               # NotificationsService – logs events
├── orders/
│   └── orders.controller.ts    # □ MODIFIED – now uses
NotificationsService          # OrdersController – POST/GET /orders
```

```

|   |—— orders.module.ts           # OrdersModule – imports
NotificationsModule
|   |—— orders.service.ts         # OrdersService – saves to DB + notifies
|   |   |—— dto/
|   |       |—— create-order.dto.ts # DTO for creating orders
|   |—— receipts/
NotificationsService
|   |—— receipts.module.ts         # ReceiptsModule – imports
NotificationsModule
|   |—— receipts.controller.ts     # ReceiptsController (from TP02)
|   |—— receipts.service.ts        # ReceiptsService – now notifies on create
|   |   |—— dto/
|   |       |—— create-receipt.dto.ts # Create DTO (from TP02)
|   |       |—— update-receipt.dto.ts # Update DTO (from TP02)

```

□ Key Concepts Before We Start

What is Dependency Injection (DI)?

Dependency Injection is a design pattern where a class **does NOT create its own dependencies**. Instead, the NestJS **IoC (Inversion of Control) container** creates them and **injects** them through the constructor.

Why is this important?

Without DI (Bad)	With DI (Good)
<code>new NotificationsService()</code> — you create it yourself	<code>constructor(private notifications: NotificationsService)</code> — NestJS creates and injects it
Tightly coupled — hard to test, hard to swap	Loosely coupled — easy to test with mocks, easy to swap implementations
If constructor changes, ALL files must update	NestJS handles it — your code doesn't care how it's built

What is a Circular Dependency?

A **circular dependency** happens when **Service A depends on Service B**, AND **Service B depends on Service A**. NestJS cannot figure out which one to create first — it's like a chicken-and-egg problem.

```

OrdersService → needs NotificationsService
NotificationsService → needs OrdersService
    □ WHO gets created first???

```

NestJS will crash with: "A circular dependency has been detected"

▣ Part A — Understand DI by Building Provider Chains

Step 0: Create the Notifications Module

First, we generate a new module and service using the NestJS CLI:

```
nest g module notifications
nest g service notifications
```

This creates two files:

- `src/notifications/notifications.module.ts`
- `src/notifications/notifications.service.ts`

Step 1: Create NotificationsService (Simple Provider)

File: `src/notifications/notifications.service.ts` (initial version)

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class NotificationsService {
  notify(event: string, payload: any) {
    console.log(`[NOTIFY] ${event}`, payload);
    return { ok: true };
  }
}
```

Line-by-line explanation:

Line	Explanation
<code>@Injectable()</code>	This decorator marks the class as a NestJS provider . It tells the IoC container: "you can create instances of this class and inject it into other classes." Without <code>@Injectable()</code> , NestJS would throw an error when another service tries to use it.

Line	Explanation
<code>notify(event: string, payload: any)</code>	A method that takes an event name (like <code>'receipt_created'</code>) and a payload (data about the event). For this lab, it simply logs to the console. In a real app, this might send emails, push notifications, or publish to a message queue.
<code>console.log(`[NOTIFY] \${event}`, payload)`</code>	Uses template literals to log the event. <code>\${event}</code> is string interpolation — it inserts the value of <code>event</code> into the string. The <code>[NOTIFY]</code> prefix makes it easy to find in logs.
<code>return { ok: true }</code>	Returns a simple confirmation object. In production, you might return a message ID or delivery status.

Teacher may ask:

- **What does `@Injectable()` do?** — It marks the class as a provider that the NestJS DI container can manage. Without it, NestJS cannot create or inject this class.
- **Can a class without `@Injectable()` be injected?** — No. NestJS needs this decorator to read metadata (constructor parameter types) for dependency resolution.
- **Is `NotificationsService` a singleton?** — Yes, by default. NestJS creates ONE instance and shares it everywhere. This is the default `SINGLETON` scope.

Step 1b: Register NotificationsModule and Export the Service

For other modules to use `NotificationsService`, we must:

1. Add `NotificationsService` to `exports[]` in `NotificationsModule`
2. Import `NotificationsModule` in `AppModule`

File: `src/notifications/notifications.module.ts` (initial version)

```
import { Module } from '@nestjs/common';
import { NotificationsService } from './notifications.service';

@Module({
  providers: [NotificationsService],
  exports: [NotificationsService], // □ Makes it available to other modules
})
export class NotificationsModule {}
```

Why `exports: [NotificationsService]` is required:

In NestJS, providers are **private to their module by default**. Only the module that declares a provider can use it. If `ReceiptsModule` wants to inject `NotificationsService`, then `NotificationsModule` MUST export it. Without `exports`, you get:

```
Error: Nest can't resolve dependencies of ReceiptsService (?).
Please make sure that the argument NotificationsService is available in
ReceiptsModule.
```

The rule:

- Module A wants a provider from Module B → Module A must `imports: [ModuleB]`
- Module B must `exports: [TheProvider]` to make it visible

File: `src/app.module.ts` (updated)

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { ReceiptsModule } from './receipts/receipts.module';
import { NotificationsModule } from './notifications/notifications.module';
import { OrdersModule } from './orders/orders.module';
import { CoreModule } from './core/core.module';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: process.env.DB_HOST || 'localhost',
      port: parseInt(process.env.DB_PORT || '5433', 10),
      username: process.env.DB_USERNAME || 'postgres',
      password: process.env.DB_PASSWORD || 'postgres',
      database: process.env.DB_NAME || 'tp02',
      autoLoadEntities: true,
      synchronize: true,
    }),
    ReceiptsModule,
    NotificationsModule, // □ Added for Part A
    OrdersModule,        // □ Added for Part B
    CoreModule,           // □ Added for Part D
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Teacher may ask:

- Why do we import `NotificationsModule` into `AppModule`? — To make sure the

`NotificationsModule` is part of the application's module graph. NestJS only knows about modules that are imported.

- **What is the module graph?** — A tree of all modules in the application, starting from `AppModule`. NestJS uses this graph to resolve dependencies.

Step 2: Inject NotificationsService into ReceiptsService

Now we use DI to inject `NotificationsService` into `ReceiptsService`, so that every time a receipt is created, a notification is triggered.

File: `src/receipts/receipts.service.ts` (updated)

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Receipt } from 'src/database/entities/receipts.entity';
import { CreateReceiptDto } from '../dto/create-receipt.dto';
import { UpdateReceiptDto } from '../dto/update-receipt.dto';
import { NotificationsService } from 'src/notifications/notifications.service'; // NEW import

@Injectable()
export class ReceiptsService {
  constructor(
    @InjectRepository(Receipt)
    private readonly receiptRepo: Repository<Receipt>,
    private readonly notifications: NotificationsService, // DI – NestJS injects this
  ) {}

  async findAll() {
    return this.receiptRepo.find({ order: { issuedAt: 'DESC' } });
  }

  async findOne(receiptId: string) {
    const receipt = await this.receiptRepo.findOne({ where: { receiptId } });
    if (!receipt) throw new NotFoundException('Receipt not found');
    return receipt;
  }

  async create(dto: CreateReceiptDto) {
    const receipt = this.receiptRepo.create({
      issuedAt: new Date(dto.issuedAt),
      name: dto.name,
      price: dto.price,
    });
    const saved = await this.receiptRepo.save(receipt);
```

```
// NEW – Notify after receipt is created
this.notifications.notify('receipt_created', {
  receiptId: saved.receiptId,
  price: saved.price,
});

return saved;
}

async update(receiptId: string, dto: UpdateReceiptDto) {
  const receipt = await this.findOne(receiptId);
  if (dto.issuedAt !== undefined) receipt.issuedAt = new Date(dto.issuedAt);
  if (dto.name !== undefined) receipt.name = dto.name;
  if (dto.price !== undefined) receipt.price = dto.price;
  return this.receiptRepo.save(receipt);
}

async remove(receiptId: string) {
  const receipt = await this.findOne(receiptId);
  await this.receiptRepo.remove(receipt);
  return { deleted: true, receiptId };
}
}
```

What changed:

1. Added `import { NotificationsService }` — to reference the class type
2. Added `private readonly notifications: NotificationsService` in the constructor — NestJS will inject it automatically
3. Called `this.notifications.notify(...)` inside `create()` — fires a notification event after saving

How the DI chain works:

```
ReceiptsService
├── @InjectRepository(Receipt) → Repository<Receipt> (provided by
TypeOrmModule.forFeature)
└── NotificationsService (provided by
NotificationsModule)
```

NestJS resolves ALL constructor dependencies before creating `ReceiptsService`. If any dependency is missing, the app crashes at startup — NOT at runtime.

Step 2b: Update ReceiptsModule to Import NotificationsModule

File: `src/receipts/receipts.module.ts` (updated)

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { ReceiptsController } from './receipts.controller';
import { ReceiptsService } from './receipts.service';
import { Receipt } from 'src/database/entities/receipts.entity';
import { NotificationsModule } from 'src/notifications/notifications.module'; // ✨ NEW

@Module({
  imports: [
    TypeOrmModule.forFeature([Receipt]),
    NotificationsModule, // ✨ This gives ReceiptsService access to
    NotificationsService
  ],
  controllers: [ReceiptsController],
  providers: [ReceiptsService],
})
export class ReceiptsModule {}
```

Why is `imports: [NotificationsModule]` needed here?

Because `ReceiptsService` (inside `ReceiptsModule`) wants to inject `NotificationsService` (from `NotificationsModule`). Without this import, NestJS cannot find `NotificationsService` in `ReceiptsModule`'s scope.

The dependency chain:

```
ReceiptsModule
  imports: [NotificationsModule] → NotificationsModule exports:
  [NotificationsService]
                                ✨
                                NotificationsService is now visible inside
ReceiptsModule
                                ✨
                                ReceiptsService can inject NotificationsService ✨
```

Teacher may ask:

- **What happens if I forget `imports: [NotificationsModule]` in `ReceiptsModule`?** — NestJS throws at startup: "Nest can't resolve dependencies of ReceiptsService. Please make sure that NotificationsService is available in ReceiptsModule."
- **What happens if I forget `exports: [NotificationsService]` in `NotificationsModule`?** — Same error. The provider exists but is private. Other modules cannot see it.

- **Can I import a service directly without importing the module?** — No. NestJS uses module-based scoping. You must import the **module**, not the service file. The `import` statement at the top of the file is just TypeScript — NestJS needs the `@Module({ imports: [...] })` decorator.

□ Checkpoint A

After completing Part A:

- `POST /receipts` creates a receipt in PostgreSQL AND prints `[NOTIFY] receipt_created { receiptId: '...', price: 12.99 }` in the server console
- This proves that **DI is working** — `ReceiptsService` successfully uses `NotificationsService` without creating it manually

□ Part B — Create a Circular Dependency (on purpose)

Step 3: Inject NotificationsService into OrdersService

We also want orders to send notifications. First, we need the `Order` entity, DTO, controller, service, and module.

File: `src/database/entities/order.entity.ts`

```
import { Column, Entity, PrimaryGeneratedColumn, CreateDateColumn } from 'typeorm';

@Entity()
export class Order {
  @PrimaryGeneratedColumn('uuid')
  orderId: string;
  // □ Auto-generated UUID primary key, same as Receipt

  @Column()
  item: string;
  // □ Name of the ordered item (e.g., "Laptop")

  @Column({ default: 1 })
  quantity: number;
  // □ How many of this item. Defaults to 1 if not provided.
  // { default: 1 } tells PostgreSQL: if no value is inserted, use 1.

  @Column({ type: 'decimal', precision: 10, scale: 2 })
  price: number;
  // □ Same as Receipt – decimal for exact money values
```

```

@CreateDateColumn()
createdAt: Date;
//   @CreateDateColumn() is a special TypeORM decorator.
//   It automatically sets the value to the current timestamp when the row is
INSERTED.
//   You NEVER set this manually – TypeORM does it for you.
//   SQL equivalent: DEFAULT CURRENT_TIMESTAMP
}

```

Teacher may ask:

- **What is `@CreateDateColumn()`?** — A decorator that automatically sets the column value to the current date/time when a new row is inserted. You don't pass it in your DTO or service — TypeORM handles it.
- **What is `{ default: 1 }` in `@Column()`?** — Sets a default value at the database level. If the INSERT query doesn't include `quantity`, PostgreSQL uses `1`.
- **Difference between `@CreateDateColumn()` and `@Column({ type: 'timestamp' })`?** — `@CreateDateColumn()` auto-fills on insert. `@Column({ type: 'timestamp' })` requires you to set the value manually.

File: `src/orders/dto/create-order.dto.ts`

```

import { IsNotEmpty, IsNumber, IsOptional, Min } from 'class-validator';

export class CreateOrderDto {
  @IsNotEmpty()
  item: string;
  //   Required field – cannot be empty string or missing

  @IsOptional()
  @IsNumber()
  @Min(1)
  quantity?: number;
  //   Optional – if not provided, the entity default (1) is used
  //   @Min(1) – quantity must be at least 1 (you can't order 0 items)
  //   The "?" makes it optional in TypeScript

  @IsNumber()
  @Min(0)
  price: number;
  //   Required – must be a number >= 0
}

```

Teacher may ask:

- **Why is `quantity` optional in the DTO but has no `@IsOptional()` on `item`?** — Because `item` is

required — every order must have an item name. **quantity** defaults to 1 at the database level, so the client doesn't have to send it.

- **What happens if I send { "item": "Laptop", "price": 999.99 } (no quantity)?** — Validation passes because `@IsOptional()`. The service uses `dto.quantity ?? 1` which falls back to 1.

File: `src/orders/orders.controller.ts`

```
import { Body, Controller, Get, Post } from '@nestjs/common';
import { OrdersService } from '../orders.service';
import { CreateOrderDto } from '../dto/create-order.dto';

@Controller('orders')
export class OrdersController {
  constructor(private readonly ordersService: OrdersService) {}
  // DI: NestJS injects OrdersService automatically

  @Post()
  create(@Body() dto: CreateOrderDto) {
    return this.ordersService.createOrder(dto);
    // Delegates to service. Controller stays thin – no business logic here.
  }

  @Get()
  findAll() {
    return this.ordersService.findAll();
    // Returns all orders from the database
  }
}
```

Teacher may ask:

- **Why is there no `@UseGuards(ApiKeyGuard)` on `OrdersController`?** — This is a design choice for the lab. The receipts endpoint requires an API key, but orders doesn't. In production, you'd protect both.
- **Why only POST and GET (no PATCH, DELETE)?** — The lab focuses on DI, not CRUD. We only need enough routes to test the notification feature.

File: `src/orders/orders.service.ts`

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Order } from '../database/entities/order.entity';
import { CreateOrderDto } from '../dto/create-order.dto';
import { NotificationsService } from '../notifications/notifications.service'; //
```

```

@Injectable()
export class OrdersService {
  constructor(
    @InjectRepository(Order)
    private readonly orderRepo: Repository<Order>,
    private readonly notifications: NotificationsService, // DI
  ) {}

  async createOrder(dto: CreateOrderDto) {
    const order = this.orderRepo.create({
      item: dto.item,
      quantity: dto.quantity ?? 1,
      // Nullish coalescing: if dto.quantity is null or undefined, use 1.
      // This matches the entity's { default: 1 } but applies at the application
      level.
      price: dto.price,
    });

    const saved = await this.orderRepo.save(order);
    // INSERT INTO order ("item", "quantity", "price") VALUES (...) RETURNING *
    // PostgreSQL auto-generates orderId (UUID) and createdAt (timestamp)

    this.notifications.notify('order_created', {
      orderId: saved.orderId,
      item: saved.item,
      price: saved.price,
    });
    // After saving, send a notification event.
    // This logs: [CORE EVENT] order_created { orderId: '...', item: 'Laptop',
    price: 999.99 }

    return saved;
  }

  async findAll() {
    return this.orderRepo.find({ order: { createdAt: 'DESC' } });
    // SELECT * FROM "order" ORDER BY "createdAt" DESC
    // Returns all orders, newest first
  }
}

```

Key point: **dto.quantity ?? 1** explained:

- **??** is the **nullish coalescing operator**
- It returns the right-hand side ONLY if the left-hand side is **null** or **undefined**
- If client sends { "item": "Laptop", "price": 999 } → **dto.quantity** is **undefined** → uses **1**
- If client sends { "item": "Laptop", "quantity": 3, "price": 999 } → **dto.quantity** is **3** → uses **3**
- This is different from **||** which also triggers on **0** or **""** (which might be valid values)

File: `src/orders/orders.module.ts`

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { OrdersController } from './orders.controller';
import { OrdersService } from './orders.service';
import { NotificationsModule } from 'src/notifications/notifications.module'; // ❏
import { Order } from 'src/database/entities/order.entity';

@Module({
  imports: [
    TypeOrmModule.forFeature([Order]), // Register Order repository
    NotificationsModule, // ❏ Import to use NotificationsService
  ],
  controllers: [OrdersController],
  providers: [OrdersService],
  exports: [OrdersService], // Export so other modules can use
  OrdersService
})
export class OrdersModule {}
```

Teacher may ask:

- **Why `exports: [OrdersService]`?** — In Part B (step 4), `NotificationsModule` will need to inject `OrdersService`. Exporting makes it available.

❏ Checkpoint B (before circular dependency)

At this point:

- `POST /orders` saves to PostgreSQL AND prints `[NOTIFY] order_created ...`
- `POST /receipts` saves to PostgreSQL AND prints `[NOTIFY] receipt_created ...`
- Both services successfully inject `NotificationsService` via DI

The DI graph so far (NO circular dependency yet):

```
ReceiptsModule —imports—> NotificationsModule
OrdersModule   —imports—> NotificationsModule

ReceiptsService —injects—> NotificationsService ❏
OrdersService   —injects—> NotificationsService ❏
```

All arrows point ONE direction → no cycle → no problem.

Step 4: Create the Circular Dependency (intentionally)

Now we intentionally make **NotificationsService** depend BACK on **OrdersService**:

File: **src/notifications/notifications.service.ts** (BROKEN version — step 4)

```
import { Injectable } from '@nestjs/common';
import { OrdersService } from 'src/orders/orders.service'; // ❌

@Injectable()
export class NotificationsService {
  constructor(private readonly ordersService: OrdersService) {} // ❌ creates a circle

  notify(event: string, payload: any) {
    console.log(`[NOTIFY] ${event}`, payload);
    return { ok: true };
  }
}
```

File: **src/notifications/notifications.module.ts** (BROKEN version — step 4)

```
import { Module } from '@nestjs/common';
import { NotificationsService } from './notifications.service';
import { OrdersModule } from 'src/orders/orders.module'; // ❌

@Module({
  imports: [OrdersModule], // ❌ Now module graph is circular too!
  providers: [NotificationsService],
  exports: [NotificationsService],
})
export class NotificationsModule {}
```

The circular dependency diagram:

```
OrdersModule —imports—> NotificationsModule
                        |
NotificationsModule —imports—> OrdersModule
                        |
                    ❌ CYCLE DETECTED!
```

```
OrdersService —injects—> NotificationsService
                        |
NotificationsService —injects—> OrdersService
                        |
                    ❌ CYCLE DETECTED!
```

What happens when you run the app:

```
ERROR [ExceptionHandler] Nest cannot create the NotificationsModule instance.  
A circular dependency has been detected between modules.  
NotificationsModule → OrdersModule → NotificationsModule
```

Why NestJS crashes:

1. NestJS tries to create `OrdersModule`
2. `OrdersModule` needs `NotificationsModule` (because `imports: [NotificationsModule]`)
3. So NestJS tries to create `NotificationsModule` first
4. But `NotificationsModule` needs `OrdersModule` (because `imports: [OrdersModule]`)
5. NestJS needs to create `OrdersModule` first... but that's what we started with!
6. Infinite loop → NestJS detects this and throws an error

Teacher may ask:

- **Why is circular dependency bad?** — The DI container cannot determine the creation order. It's like saying "A must be created before B" AND "B must be created before A" — both cannot be true.
- **Does a circular dependency only happen between 2 modules?** — No. It can be `A → B → C → A` (three modules forming a cycle).
- **Is this a compile-time or runtime error?** — Runtime. TypeScript compiles fine. The error happens when NestJS bootstraps and tries to build the module graph.

□ Part C — Fix Circular Dependency (Method 1: `forwardRef`)

Step 5: Fix Module Circular Dependency with `forwardRef()`

`forwardRef()` tells NestJS: "don't resolve this import right now — wait until the full module graph is built."

File: `src/orders/orders.module.ts` (fixed with `forwardRef`)

```
import { forwardRef, Module } from '@nestjs/common';  
import { NotificationsModule } from 'src/notifications/notifications.module';  
  
@Module({  
  imports: [  
    forwardRef(() => NotificationsModule), // □ Delayed resolution
```

```

],
controllers: [OrdersController],
providers: [OrdersService],
})
export class OrdersModule {}

```

File: `src/notifications/notifications.module.ts` (fixed with `forwardRef`)

```

import { forwardRef, Module } from '@nestjs/common';
import { OrdersModule } from 'src/orders/orders.module';

@Module({
  imports: [forwardRef(() => OrdersModule)], // ⚠ Delayed resolution
  providers: [NotificationsService],
  exports: [NotificationsService],
})
export class NotificationsModule {}

```

How `forwardRef()` works:

- Normal import: `imports: [NotificationsModule]` → NestJS tries to resolve `NotificationsModule` **immediately**
- `forwardRef`: `imports: [forwardRef(() => NotificationsModule)]` → NestJS stores a **callback function** and calls it **later**, after the module graph is partially built
- The arrow function `() => NotificationsModule` delays the reference — JavaScript doesn't evaluate it until NestJS calls it
- This breaks the "who comes first" deadlock

Step 6: Fix Provider Injection with `@Inject(forwardRef())`

Fixing modules alone is not enough — we also need to fix the **service-level** circular injection.

File: `src/notifications/notifications.service.ts` (fixed with `forwardRef`)

```

import { forwardRef, Inject, Injectable } from '@nestjs/common';
import { OrdersService } from 'src/orders/orders.service';

@Injectable()
export class NotificationsService {
  constructor(
    @Inject(forwardRef(() => OrdersService)) // ⚠ Delayed injection
    private readonly ordersService: OrdersService,
  ) {}

  notify(event: string, payload: any) {

```

```

    console.log(`[NOTIFY] ${event}`, payload);
    return { ok: true };
  }
}

```

Why is `@Inject(forwardRef(...))` needed at the service level too?

There are **two levels** of circular dependency:

Level	Problem	Fix
Module level	Module A imports Module B, Module B imports Module A	<code>forwardRef()</code> in <code>imports: [...]</code> of BOTH modules
Provider level	Service A injects Service B, Service B injects Service A	<code>@Inject(forwardRef)</code> ⇒ <code>ServiceB</code> in the constructor

You must fix BOTH levels for it to work.

Teacher may ask:

- **What is `forwardRef()`?** — A NestJS utility function that wraps a reference in a callback to delay its resolution. NestJS evaluates the callback during a later phase of module initialization.
- **Why do we need `forwardRef` on BOTH sides?** — Because both modules/services reference each other. If only one side uses `forwardRef`, the other side still tries to resolve immediately and fails.
- **Is `forwardRef` a good solution?** — It works but is a code smell. It means your architecture has a design problem. Method 2 (Part D) is the better approach.
- **When should you use `forwardRef`?** — Only as a temporary fix or when the circular dependency is genuinely unavoidable (which is rare).

□ Checkpoint C

After applying `forwardRef()`:

- The app starts without errors □
- `POST /orders` still logs notifications □
- `POST /receipts` still logs notifications □

But the code has a smell — `NotificationsService` depends directly on `OrdersService`, which defeats the purpose of having a generic notification system.

□ Part D — Fix Circular Dependency (Method 2: Refactor with Abstraction)

This is the **proper architectural solution**. Instead of `NotificationsService` calling `OrdersService` directly, we introduce an **abstraction layer** using a token-based provider.

Step 7a: Create the Core Module with Token-Based Provider

File: `src/core/tokens.ts`

```
export const EVENT_PUBLISHER = Symbol('EVENT_PUBLISHER');
```

What is happening here:

- `Symbol('EVENT_PUBLISHER')` creates a **unique token** — a special JavaScript value that is guaranteed to be unique
- This token acts as an **identifier** for the dependency injection system
- Instead of injecting a class (like `NotificationsService`), we inject a **token** that can be bound to any implementation
- `Symbol` is like a unique key that can never clash with another key, even if they have the same description string

Why use `Symbol` instead of a string?

Token Type	Example	Risk
String	<code>'EVENT_PUBLISHER'</code>	Two developers might accidentally use the same string in different places → conflict
Symbol	<code>Symbol('EVENT_PUBLISHER')</code>	Every <code>Symbol()</code> call creates a globally unique value → impossible to conflict

File: `src/core/core.module.ts`

```
import { Module } from '@nestjs/common';
import { EVENT_PUBLISHER } from '../tokens';

@Module({
  providers: [
    {
      provide: EVENT_PUBLISHER,
      // □ The injection token – when someone asks for EVENT_PUBLISHER, give them this
```

```

value
  useValue: {
    publish: (event: string, payload: any) => {
      console.log(`[CORE EVENT] ${event}`, payload);
    },
  },
  // □ The actual implementation – a simple object with a publish() method
  //   useValue means "use this exact object as the provider"
},
exports: [EVENT_PUBLISHER],
// □ Export the token so other modules can import CoreModule and inject
EVENT_PUBLISHER
})
export class CoreModule {}

```

What is the `{ provide, useValue }` pattern?

This is a **custom provider** in NestJS. Instead of the standard shorthand `providers: [SomeService]`, we define exactly what to provide and how:

Provider Pattern	Use Case	Example
<code>providers: [MyService]</code>	Standard — NestJS creates the class instance	<code>providers: [NotificationsService]</code>
<code>{ provide: TOKEN, useValue: obj }</code>	Provide a specific value/object	<code>{ provide: EVENT_PUBLISHER, useValue: { publish: ... } }</code>
<code>{ provide: TOKEN, useClass: MyClass }</code>	Provide a class (NestJS creates it)	<code>{ provide: 'LOGGER', useClass: WinstonLogger }</code>
<code>{ provide: TOKEN, useFactory: fn }</code>	Provide the return value of a factory function	<code>{ provide: 'CONFIG', useFactory: () => loadConfig() }</code>

Teacher may ask:

- **What is a token-based provider?** — Instead of using a class as the injection key, you use a `Symbol` or string. This decouples the consumer from the implementation.
- **What does `useValue` do?** — Provides a specific object as-is. NestJS doesn't create anything — it just passes this exact object when someone injects `EVENT_PUBLISHER`.
- **Why not use `useClass`?** — `useValue` is simpler for this case since we just need a function. `useClass` would require a separate class file.
- **Could we swap implementations?** — Yes! That's the whole point. We could change `useValue` to point to a Redis publisher, Kafka producer, or email service — without changing `NotificationsService`.

Step 7b: Update NotificationsService to Use the Token

File: `src/notifications/notifications.service.ts` (final version — no circular dependency)

```
import { Inject, Injectable } from '@nestjs/common';
import { EVENT_PUBLISHER } from 'src/core/tokens';

type EventPublisher = { publish: (event: string, payload: any) => void };
//   TypeScript type definition – describes the SHAPE of the publisher object
//   Any object with a publish(event, payload) method satisfies this type
//   This is like an interface but simpler (type alias)

@Injectable()
export class NotificationsService {
  constructor(
    @Inject(EVENT_PUBLISHER)
    private readonly publisher: EventPublisher,
    //   @Inject(EVENT_PUBLISHER) tells NestJS: "find the provider registered with the
    //   EVENT_PUBLISHER token and inject it here"
    //   NestJS looks up CoreModule's providers and finds the { provide:
EVENT_PUBLISHER, useValue: {...} }
    //   Then injects that object here
  ) {}

  notify(event: string, payload: any) {
    this.publisher.publish(event, payload);
    //   Calls the publish() method of the injected publisher
    //   Since CoreModule provides { publish: (event, payload) => console.log(...) },
    //   this will log: [CORE EVENT] receipt_created { receiptId: '...', price: 12.99
  }

  return { ok: true };
}
```

What changed compared to the broken version:

Before (circular)	After (refactored)
<code>import { OrdersService } — depends on a specific service</code>	<code>import { EVENT_PUBLISHER } — depends on an abstract token</code>
<code>constructor(private ordersService: OrdersService) — needs OrdersService instance</code>	<code>constructor(@Inject(EVENT_PUBLISHER) private publisher: EventPublisher) — needs any publisher</code>
<code>NotificationsModule imports OrdersModule — creates a cycle</code>	<code>NotificationsModule imports CoreModule — no cycle, CoreModule depends on nothing</code>

Step 7c: Update NotificationsModule

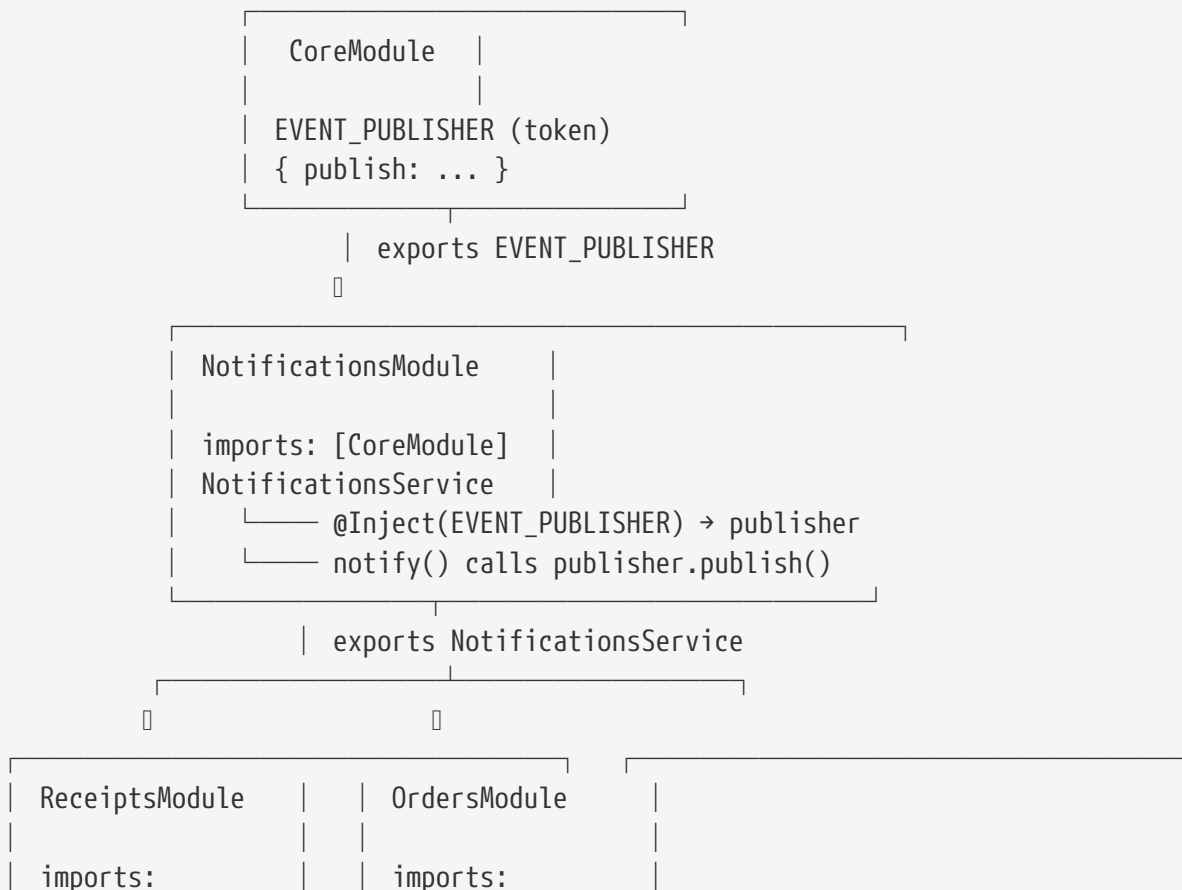
File: `src/notifications/notifications.module.ts` (final version)

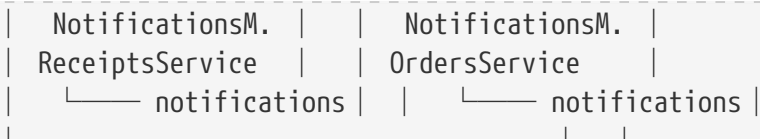
```
import { Module } from '@nestjs/common';
import { NotificationsService } from './notifications.service';
import { CoreModule } from 'src/core/core.module';

@Module({
  imports: [CoreModule],          // ◻ Imports CoreModule (provides
  EVENT_PUBLISHER)
  providers: [NotificationsService], // ◻ NotificationsService can now inject
  EVENT_PUBLISHER
  exports: [NotificationsService], // ◻ Other modules can still use
  NotificationsService
})
export class NotificationsModule {}
```

No more **forwardRef**! No more importing **OrdersModule**!

◻ Final Architecture (No Circular Dependency)





All arrows point UPWARD — no cycles!

- **CoreModule** depends on NOTHING — it's a standalone utility
- **NotificationsModule** depends only on **CoreModule**
- **ReceiptsModule** and **OrdersModule** depend on **NotificationsModule**
- Nobody loops back

□ Summary: Comparison of All Methods

Aspect	Part B (Broken)	Part C (forwardRef)	Part D (Refactored)
Circular dependency?	□ Yes — app crashes	□□ Resolved with forwardRef	□ None — clean architecture
Code complexity	Simple but broken	Added forwardRef wrappers	Added CoreModule + token
Coupling	Tight — Notifications □ Orders	Still tight — just hidden	Loose — Notifications only knows about a publisher interface
Testability	Hard — mocking circular deps is painful	Possible but messy	Easy — swap publisher mock in tests
Scalability	Cannot grow without more circles	More forwardRef = more mess	Add new publishers without changing services
Real-world recommendation	Never do this	Temporary fix only	□ Preferred approach

□ Testing with HTTP Requests

File: **TP03.http**

```

### Variables
@baseUrl = http://localhost:3000
@apiKey = my-secret-api-key-123

### --- Part A: Notifications via Receipts ---
  
```

```

### 1. POST /receipts - Create a Receipt
### (saves to DB + logs [CORE EVENT] receipt_created in console)
POST {{baseUrl}}/receipts
Content-Type: application/json
x-api-key: {{apiKey}}

{
  "issuedAt": "2026-03-24T10:00:00Z",
  "name": "Notebook",
  "price": 12.99
}

### 2. GET /receipts - Verify receipts were created
GET {{baseUrl}}/receipts
x-api-key: {{apiKey}}

### --- Part B/C/D: Orders with Notifications ---

### 3. POST /orders - Create an Order
### (saves to DB + logs [CORE EVENT] order_created in console)
POST {{baseUrl}}/orders
Content-Type: application/json

{
  "item": "Laptop",
  "quantity": 1,
  "price": 999.99
}

### 4. GET /orders - Verify all orders saved
GET {{baseUrl}}/orders

```

Expected console output after POST /receipts:

```
[CORE EVENT] receipt_created { receiptId: 'a1b2c3d4-...', price: '12.99' }
```

Expected console output after POST /orders:

```
[CORE EVENT] order_created { orderId: 'e5f6g7h8-...', item: 'Laptop', price: '999.99'
}
```

□ Potential Teacher Questions & Answers

About DI

1. What is Dependency Injection?

A design pattern where dependencies are provided (injected) by an external container instead of being created by the class itself. In NestJS, the IoC container reads constructor parameter types and automatically provides instances.

2. What is the IoC Container?

Inversion of Control container — it's the "brain" of NestJS that manages creating, storing, and injecting all providers. When the app starts, the container scans all modules, reads all providers and their constructor dependencies, and builds a dependency graph.

3. What are the three steps to make DI work?

1. Mark the class with `@Injectable()` — registers it with the container
2. Add it to `providers: [...]` in its module — tells the module "I own this provider"
3. Add the type in the constructor — NestJS reads the type and injects the matching provider

4. What is the difference between `providers` and `exports` in `@Module()`?

`providers` registers the class within THIS module's scope. `exports` makes it visible to OTHER modules that import this module. A provider that is not exported is private.

About Circular Dependency

What is a circular dependency?

When two or more modules/services depend on each other, forming a cycle. NestJS cannot determine which one to create first, so it throws an error.

How does `forwardRef()` fix circular dependency?

It wraps the reference in a callback function `() => ClassName`. NestJS stores this callback and evaluates it later, after the module graph is partially constructed. This breaks the immediate resolution deadlock.

Why is `forwardRef()` not the best solution?

It hides the architectural problem without solving it. The circular dependency still exists logically. As the project grows, more `forwardRef` calls make the code harder to understand and maintain.

About Part D (Refactoring)

What is a token-based provider?

A provider identified by a `Symbol` or string instead of a class reference. You use `@Inject(TOKEN)` instead of just declaring the type. This allows injecting abstract contracts instead of concrete implementations.

What is the benefit of the CoreModule approach?

It creates a one-way dependency flow. `NotificationsService` doesn't know about `OrdersService` at all — it only knows about an `EventPublisher` interface. This follows the **Dependency Inversion**

Principle (the "D" in SOLID): depend on abstractions, not concrete implementations.

Can we swap the `EVENT_PUBLISHER` implementation?

Yes! We just change `useValue` in `CoreModule`. For example, we could swap `console.log` with a Redis publisher, a Kafka producer, or a WebSocket broadcaster — without changing `NotificationsService` or any other service that uses it.

What is `Symbol()` in JavaScript?

A primitive type that creates a globally unique value. `Symbol('A') !== Symbol('A')` — even with the same description string, every `Symbol()` call creates a new unique value. It's perfect for injection tokens because it prevents accidental collisions.